
PGD-NO: A Neural Operator with Precomputed Geometry Decomposition for 3D Million-scale Physics Simulations

Weiheng Zhong^{1 2} Jing Bi² Victor Oancea² Hadi Meidani^{1 3}

Abstract

While neural PDE solvers have demonstrated significant potential for accelerating engineering simulations, existing architectures remain constrained by high memory consumption and the “single-node bottleneck,” where the maximum processable mesh resolution is strictly limited by the VRAM of a single compute unit. To address these challenges, we propose PGD-NO, a neural operator with Precomputed Geometry Decomposition, that relocates the computational overhead of geometric encoding to a deterministic pre-computation phase. By utilizing an iterative geometry decomposition algorithm to extract “geometry tokens,” our model decouples feature extraction from solution querying. This architecture enables linear memory scalability, allowing high-fidelity learning on meshes exceeding 10 million nodes—a scale where existing architectures typically encounter memory exhaustion. PGD-NO demonstrates competitive predictive accuracy across diverse industrial benchmarks and provides intrinsic interpretability through attention mechanisms. By effectively overcoming traditional mesh-size constraints, PGD-NO offers a robust and efficient solution for the next generation of large-scale, high-fidelity industrial design applications. Our codes and datasets are available at <https://github.com/WeihengZ/PGD-NO>.

1. Introduction

The numerical solution of partial differential equations (PDEs) is a cornerstone of computational science and engineering, facilitating evaluations (Jagota et al., 2013), sensitivity analysis (Hisada, 1995), and uncertainty quantification (Zhong & Meidani, 2023) in industrial design. Traditional discretization-based methods—such as the finite element method (FEM) (Jagota et al., 2013) and finite volume method (FVM) (Moukalled et al., 2015)—are highly refined, offering significant accuracy and robustness across diverse domains. Nevertheless, these conventional approaches often involve prohibitive computational costs, particularly for large-scale, high-fidelity simulations (Lu et al., 2021a). To mitigate these demands, deep learning architectures have been investigated as efficient surrogates for numerical methods, called neural PDE solvers (Lu et al., 2021b; Li et al., 2021; Wu et al., 2024). By training on simulation data, these neural solvers learn to approximate the mapping between inputs and outputs of the traditional methods, thereby significantly accelerating the manufacturing and design process.

Industrial applications typically involve large, complex geometries, requiring models to efficiently capture the underlying physical features (Wu et al., 2024). DeepONet (Lu et al., 2021b) was initially applied to industrial design because many geometries are governed by low-dimensional parameter vectors for manufacturing purposes. However, recent studies indicate that the performance of DeepONet diminishes as geometric complexity increases (Li et al., 2021). Simultaneously, various deep learning models have been proposed for freeform geometries. Although these methods show promise in benchmark problems, they struggle with real-world industrial tasks involving millions of mesh points (Luo et al., 2025). This is because training on such massive datasets requires significant GPU memory, which often exceeds the capacity of standard hardware.

Existing neural PDE solvers for freeform geometries can be categorized into graph-based, grid-based, and point-based models, depending on how they represent PDE solutions on a mesh. Graph-based models (Anandkumar et al., 2020; Velasco et al., 2024; Li et al., 2020; Gladstone et al., 2024) may fail to capture complex geometries if they rely on sparse mesh graphs; conversely, they can rapidly exhaust GPU

¹Department of Civil and Environmental Engineering, Grainger College of Engineering, University of Illinois Urbana-Champaign, Urbana, IL, US ²Dassault Systemes Americas, Johnston, RI, US ³Siebel School of Computing and Data Science, Grainger College of Engineering, University of Illinois Urbana-Champaign, Urbana, IL, US. Correspondence to: Weiheng Zhong <weiheng4@illinois.edu>.

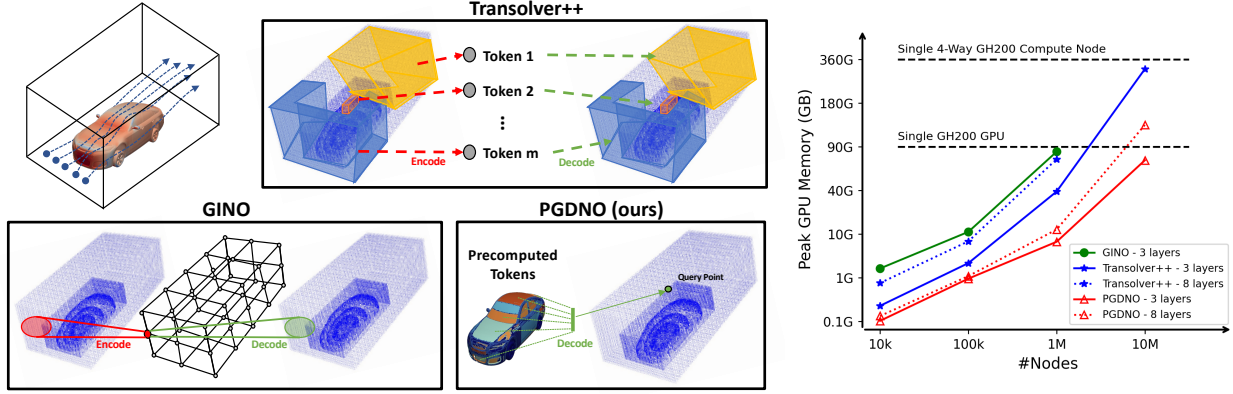


Figure 1. Architectural comparison and memory scaling. (Left) Encoding mechanisms: GINO utilizes grid-based mapping, and Transolver++ computes physics tokens via weighted nodal sums, whereas PGD-NO employs pre-computed geometry tokens to bypass intensive encoding. (Right) Peak GPU memory on a GH200 system: PGD-NO maintains the lowest footprint, enabling high-resolution simulations (up to 10M nodes) that exceed the capacity of traditional grid-based or graph-based neural operators.

memory if they utilize radius-based methods to synthesize edges. Grid-based models (Sahebi et al., 2025), such as GINO (Li et al., 2023a), are also memory-intensive, as capturing complex geometries requires mapping point features onto high-resolution grids. In contrast, point-based methods (Kashefi & Mukerji, 2022; Park & Kang, 2026; Lee & Oh, 2024), such as Transolver (Wu et al., 2024), utilize a physics-attention mechanism to identify underlying “physics tokens,” demonstrating that complex geometric features can be captured directly from point clouds. Recently, Transolver++ (Luo et al., 2025) has extended this capability to million-scale problems through an optimized parallelism framework.

Despite its advancements, Transolver++ still faces limitations when scaled to high-fidelity PDE solving tasks. In practice, we observe significant bottlenecks in training efficiency when mesh sizes exceed 10 million nodes. While Transolver++ can theoretically scale via parallel computation, this comes at the cost of increased communication overhead. Although intra-node communication remains efficient due to NVLink (Wei et al., 2023) architecture, scaling beyond a single compute node necessitates data transfers between compute nodes via standard networking. This inter-node communication introduces substantial latency, significantly increasing the duration of each training epoch. Consequently, the practical mesh-size capacity of Transolver++ remains constrained by the resources of a single compute node. As illustrated in Figure 1, even utilizing the latest 4-way GH200 architecture with 360GB of total memory, a Transolver++ model with a reduced number of layers can barely accommodate 10M nodes for CFD simulation learning.

We consider that the GPU memory limitations of existing

methods stem from their reliance on learning-based encoding process to map complex geometries into hidden representations. In contrast, geometric features can be pre-computed using deterministic algorithms, which significantly reduces GPU memory consumption by bypassing the encoding process. Motivated by this observation, this work explores an alternative paradigm for developing neural PDE solvers. We introduce a geometry decomposition algorithm for feature extraction and propose a simple but effective decoding architecture to predict PDE solutions based on these pre-computed features. Hence, our primary contributions in this work are as follows:

- We introduce a deterministic hierarchical decomposition algorithm to extract “geometry tokens,” demonstrating that PDE solutions can be effectively learned using non-learnable geometric primitives.
- We propose the Pre-computed Geometry Decomposition Neural Operator (PGD-NO), which decouples geometry encoding from solution querying, eliminating inter-node communication for million-scale mesh processing on standard hardware.
- We evaluate our model against current state-of-the-art methods across five industrial benchmarks, including a novel, self-generated dataset of up to 12 million nodes, where our approach achieves superior performance.
- We further demonstrate that PGD-NO enables efficient training via random sampling while providing enhanced interpretability for its physical predictions

Conflict of Interest Disclosure The authors J.B. and V.O. are employed by Dassault Systèmes, and W.Z. is currently

employed by Dassault Systèmes. This paper evaluates multiple datasets, including one provided by Dassault Systèmes.

2. Related Works

2.1. Neural PDE Solver

Inspired by DeepONet, branch-trunk operators leverage the universal operator approximation theorem to map infinite-dimensional function spaces (Lu et al., 2021a). These architectures consist of a branch network encoding input functions and a trunk network processing output coordinates (Lu et al., 2021b). While extensions have addressed the curse of dimensionality (Mandl et al., 2025), uncertainty quantification (Lone et al., 2026), and computational efficiency (Zhang et al., 2025; Qiu et al., 2024), and specialized variants like GANO (Zhong & Meidani, 2025) and Point-DeepONet (Park & Kang, 2026) incorporate geometric information, these models generally rely on fixed sensor locations or global pooling. Consequently, despite improvements in expressiveness (Zhong & Meidani, 2024) and temporal encoding (He et al., 2024), branch-trunk architectures remain fundamentally limited in their ability to handle truly freeform, complex geometries.

Graph-based neural operators (GNOs) (Anandkumar et al., 2020) are designed for irregular domains by utilizing message-passing frameworks. Key developments such as MGNO (Li et al., 2020) and LNO (Wang & Wang, 2024) enable the capture of long-range interactions and expressive latent mappings. To generalize across arbitrary geometries, models like CORAL (Serrano et al., 2023), EA-GNN (Gladstone et al., 2024), and PDE-GCN (Eliasof et al., 2021) have introduced coordinate-based features and PDE-aware message passing. However, a significant drawback of these methods is the memory overhead required to store and process edge information; as the mesh density increases, the memory cost of the graph connectivity can quickly exceed the capacity of modern hardware.

Grid-based operators, primarily led by the Fourier Neural Operator (FNO) (Li et al., 2021), capture global dependencies in the frequency domain with high efficiency. Scalable variants like D-FNO (Li & Ye, 2025), alongside geometry-aware models like GINO (Li et al., 2023a) and DAFNO (Liu et al., 2023), attempt to adapt spectral methods to irregular topologies. While these models provide significant speedups (Lu et al., 2022) and incorporate physics-informed constraints (Li et al., 2024), they rely on mapping unstructured data to latent regular grids. In large-scale industrial applications, these grids must maintain an extremely high resolution to avoid significant information loss, which becomes computationally prohibitive.

Point-based operators offer the greatest flexibility for nonuniform resolutions and complex geometries by oper-

ating directly on spatial point clouds. Architectures such as GNOT (Hao et al., 2023) and Transolver (Wu et al., 2024) utilize physics-aware attention to identify underlying tokens, while frameworks like Universal Physics Transformers (UPT) (Alkin et al., 2024) and GFN (Morrison et al., 2024) ensure resolution invariance. Although these methods mitigate numerical artifacts (Zheng et al., 2024; Yu et al., 2024) and achieve better scalability (Li et al., 2023b), a clear limitation persists: while their memory complexity scales linearly with the number of nodes, the maximum node capacity remains strictly capped by the GPU memory limits of a single compute node.

2.2. Geometry Decomposition

Traditional geometric decomposition methods (Jermann et al., 2006) often rely on deterministic algorithms such as K-means clustering on mesh faces (Mamou & Ghorbel, 2009), hierarchical clustering based on curvature, or spectral clustering utilizing the Laplace-Beltrami operator (Lévy, 2006) to identify salient features. Advanced techniques such as convex decomposition (Lien & Amato, 2008) aim to break complex, non-convex volumes into a set of simpler, nearly convex parts to facilitate more efficient physical simulations. In the context of industrial design, primitive-based segmentation is frequently employed to decompose geometries into standard shapes (e.g., planes, cylinders, or spheres) that correspond to manufacturing parameters (Elrefaie et al., 2024a; Luo et al., 2025). While these classical segmentation methods offer high interpretability without requiring training data, their integration into end-to-end neural PDE solvers remains largely unexplored. Furthermore, achieving further reductions in computational complexity requires a method to identify a sparse yet representative set of geometry tokens—a capability currently absent in existing geometric segmentation algorithms.

3. Method

3.1. Geometry Token Extraction

The proposed workflow transforms a raw 3D mesh into a set of discrete, semantically meaningful physics tokens by combining graph topology with geometric feature detection.

Step (1) Graph Construction: The process begins by transforming the raw 3D mesh into a structured topological network $G = (V, E)$, where V is the set of mesh vertices and E is the set of edges.

Step (2) Sharp Edge Identification: To determine where a mesh should naturally break, the algorithm identifies sharp features using geometric gradients. The procedure evaluates the dihedral angle θ_{ij} between the normals $\mathbf{n}_a$ and $\mathbf{n}_b$ of adjacent faces sharing edge e_{ij} . If the angle satisfies: $\theta_{ij} = \arccos(\mathbf{n}_a \cdot \mathbf{n}_b) > \theta_{\text{threshold}}$, the nodes $v \in e_{ij}$ are added to

a node set of sharp edges V_{sharp} , where $\theta_{\text{threshold}}$ is the angle threshold. This acts as a structural shattering mechanism (Szuszik et al., 2026), where prominent ridges are marked for disconnection.

Step (3) Graph Partition: Once the sharp edges are identified, the partition is executed by creating a subgraph $G' = G[V \setminus V_{\text{sharp}}]$, where all sharp edge boundary are removed. This operation of node removal causes the graph to fracture into a set of independent connected components $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$. To achieve a hierarchical refinement of the mesh, the process is performed iteratively across a sequence of thresholds. We introduce a threshold decay mechanism, $\theta_{\text{threshold}}^{i+1} = \alpha \theta_{\text{threshold}}^i$ with a hyper-parameter $\alpha \in (0, 1)$, which progressively increases the sensitivity of sharp edge detection. This decay allows the algorithm to capture coarse structural parts at higher angles before isolating finer geometric details at lower angles. Any subgraph C_i that satisfies the size constraint $|C_i| > \text{min_size}$ is preserved, thereby preventing the generation of excessively small geometry tokens that could undermine computational efficiency.

Step (4) Graph Merge: To prevent the creation of excessive noise, the algorithm employs a merging logic based on proximity. For a small fragment G_{small} where $|V_{\text{small}}| < \text{threshold}$, the algorithm identifies the boundary set $\partial V_{\text{small}}$ and expands it by h hops using the adjacency relation $\text{Adj}(v)$. The merge occurs when an expanded set $V_{\text{small}}^{(h)}$ intersects with another larger graph node set V_{large} . Using this strategy, small graphs are absorbed into the most relevant neighbor, ensuring that the final set of tokens remains computationally manageable.

Step (5) Geometry token matrix: In the last step, we create the mapping matrix $S \in \mathbb{R}^{K \times N}$ representing the relationship between geometry tokens and mesh nodes, where K is the number of geometry tokens and N is the number of mesh vertices:

$$S_{ki} = \begin{cases} \frac{1}{|C_k|} & \text{if } v_i \in C_k, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

The details of the geometry token creation algorithm is shown in Algorithm 1.

3.2. Model Architecture

Upon the generation of the sparse segmentation matrix $S \in \mathbb{R}^{S \times N}$, the model integrates these precomputed geometry tokens into a deep learning framework to process the input point cloud. The input mesh, represented as a point cloud of N points with spatial coordinates in $\mathbb{R}^{N \times 3}$, is first projected into a high-dimensional feature space $\mathbf{X} \in \mathbb{R}^{N \times C}$. These features are then aggregated into the token space by computing the matrix product of the segmentation matrix

Algorithm 1 Iterative Geometry Decomposition

- 1: **Input:** Mesh PolyData $\mathcal{P}$, Coordinates $\mathbf{X}$, Angle thresholds $\Theta = \{\theta_1, \theta_2, \dots, \theta_m\}$, Min graph size ratio ρ
 - 2: **1. Graph construction:**
 - 3: Build adjacency graph $G = (V, E)$ from mesh connectivity $\mathcal{P}$
 - 4: Initialize set of subgraphs to split: $\mathcal{S} = \{G\}$
 - 5: **2. Iterative partition with sharp edges:**
 - 6: **for** $\theta \in \Theta$ **do**
 - 7: $V_{\text{sharp}} = \{v \in V \mid \text{dihedral angle at } v > \theta\}$
 - 8: $\mathcal{S}_{\text{next}} = \emptyset$
 - 9: **for** $g \in \mathcal{S}$ **do**
 - 10: $\{\mathcal{C}_1, \dots, \mathcal{C}_k\} = \text{Graph-Split}(g \setminus V_{\text{sharp}})$
 - 11: $\mathcal{S}_{\text{next}} = \mathcal{S}_{\text{next}} \cup \{\mathcal{C}_i \mid |\mathcal{C}_i| > \rho \cdot |V|\}$
 - 12: **end for**
 - 13: $\mathcal{S} = \mathcal{S}_{\text{next}}$
 - 14: **if** $\mathcal{S} = \emptyset$ **then**
 - 15: **break**
 - 16: **end if**
 - 17: **end for**
 - 18: **3. Graph remerge**
 - 19: $V_{\text{unassigned}} = V \setminus \bigcup_{g \in \mathcal{S}} V(g)$
 - 20: $\mathcal{S} = \mathcal{S} \cup \text{Graph-Split}(G[V_{\text{unassigned}}])$
 - 21: Sort $\mathcal{S}$ by size descending: $\{\mathcal{S}_{\text{large}}, \mathcal{S}_{\text{small}}\}$
 - 22: **for** $g_{\text{small}} \in \mathcal{S}_{\text{small}}$ **do**
 - 23: Perform h -hop BFS expansion in G : $V_{\text{expanded}} = \text{BFS}(V(g_{\text{small}}), h)$
 - 24: **if** $V_{\text{expanded}} \cap V(g_{\text{large}}) \neq \emptyset$ **then**
 - 25: Merge $V(g_{\text{small}})$ into the first intersecting $V(g_{\text{large}})$
 - 26: **end if**
 - 27: **end for**
 - 28: **4. Token encoding:**
 - 29: Construct sparse matrix $\mathbf{M} \in \mathbb{R}^{|\mathcal{S}| \times |V|}$
 - 30: $S_{ij} = \begin{cases} 1/|V(\mathcal{S}_i)| & \text{if } v_j \in \mathcal{S}_i \\ 0 & \text{otherwise} \end{cases}$
 - 31: **Output:** Sparse segmentation matrix $\mathbf{S}$
-

and the point features, $\mathbf{S}\mathbf{X} \in \mathbb{R}^{S \times C}$. This operation effectively down-samples the dense point information into a compact latent token representation of the geometry.

The core of the architecture consists of M successive Geometric Token Layers designed to refine the interaction between global geometric tokens and local point features. As shown in the model architecture diagram in Figure 2, the aggregated geometry token features serve as a global context that is fed back into the nodal features with an Attention Layer, which enables the network to propagate information from semantically distinct regions back to individual points.

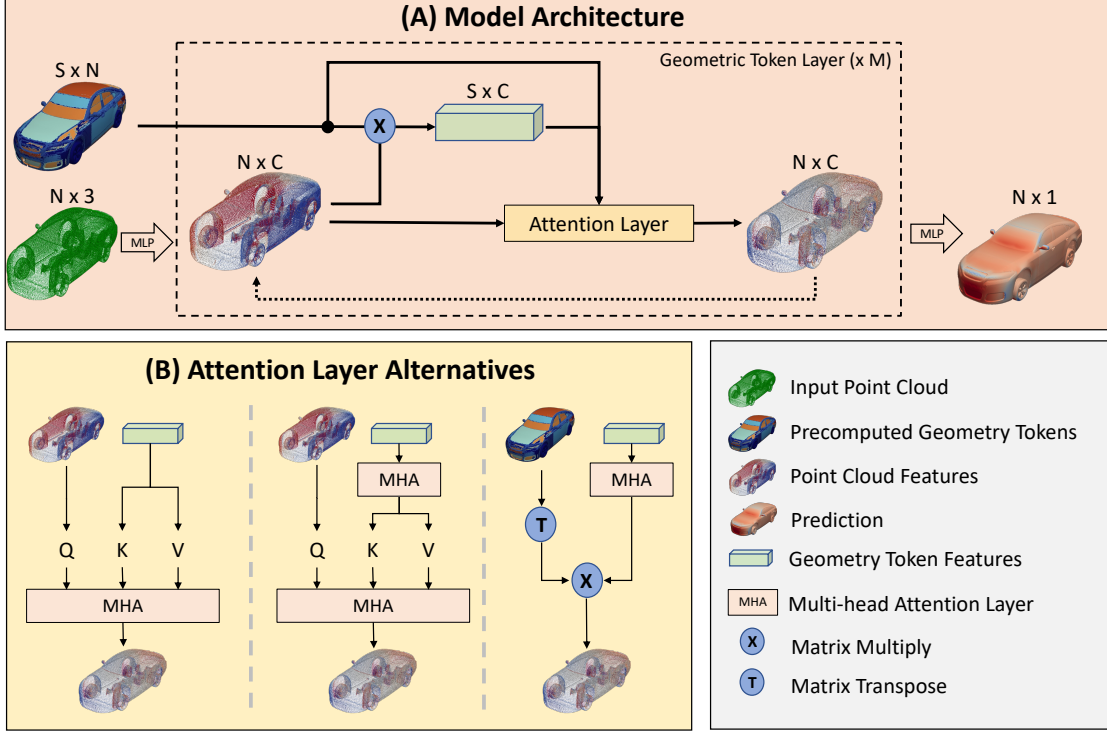


Figure 2. PGD-NO architecture and layer variants. (Top) Model pipeline: a pre-computed segmentation matrix extracts geometry tokens from the point cloud, which are refined by stacked layers to capture multi-scale physics. (Bottom Left) Layer configurations: Variants 1 and 2 use multi-head attention to project features from tokens back to nodes, while Variant 3 uses a transposed segmentation matrix for high-efficiency surface mapping.

Following the iterative refinement within the Geometric Token Layers, the final high-dimensional point features are passed through a concluding MLP to produce the target predictions.

The proposed framework introduces three architectural alternatives for the geometric token layer, varying in how they reconstruct point-wise features from the latent space. In the first variant, a multi-head cross-attention mechanism directly maps token features back to point features by utilizing the input point cloud as the Query (Q) and the geometry tokens as the Key (K) and Value (V). The second variant introduces an additional Multi-Head Attention (MHA) refinement step on the token features prior to the cross-attention mapping, allowing for more complex interactions within the latent representation. The third variant offers a high-efficiency alternative specifically for surface prediction by transposing the segmentation matrix to map features directly back to each mesh node. However, since these geometry tokens are precomputed based on specific mesh nodes, this third architecture is restricted to surface tasks and cannot be extended to volume node prediction.

3.3. Model Complexity Analysis

To establish a baseline for performance, we evaluate the computational and memory complexity of our proposed architecture against Transolver++, which represents the best state-of-the-art neural PDE solver framework in terms of efficiency. Our analysis considers a 3D PDE problem defined by N_s surface nodes, which characterize the geometry, and N_v volume nodes, representing the spatial coordinates where the Partial Differential Equation (PDE) solutions are computed.

The system’s computational complexity is primarily governed by operations performed on volume node features. While our model maintains the same order of magnitude as Transolver++, it achieves a 50% reduction in computational overhead ($O(4N_v)$ versus $O(8N_v)$) by streamlining the attention mechanism. Furthermore, by decoupling geometry extraction from solution querying, our framework significantly enhances memory efficiency. This architecture allows volume nodes to be partitioned across multiple GPUs, shifting the primary memory bottleneck to surface node operations ($O(4N_s)$), a substantial improvement over the $O(8N_v)$ bottleneck inherent in Transolver++. More details about this analysis can be found in Appendix A.

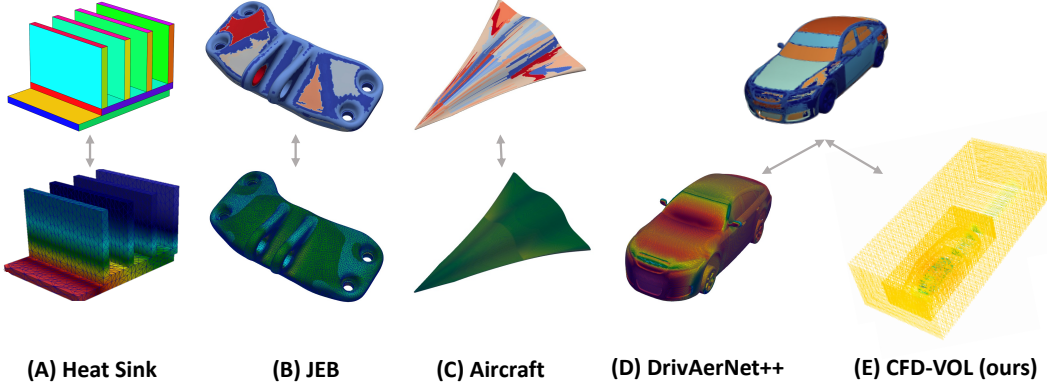


Figure 3. Visualization of 3D industrial benchmarks. (Top) Geometry tokens generated via our iterative decomposition algorithm for representative samples. (Bottom) Corresponding ground-truth physics fields. These benchmarks cover a wide spectrum of complexities—from smooth surfaces to irregular, multi-component geometries—and diverse prediction tasks, including internal volume, surface-based, and external fluid domain patterns.

4. Experiment

Data: Given that two-dimensional problems have been extensively explored in the literature of Neural PDE solvers, this study focuses exclusively on assessing model performance within the context of three-dimensional, real-world industrial applications. The evaluation is conducted across the following five benchmarks: Heat Sink (Zhong, 2025), JEB (Hong et al., 2025), DrivAerNet++ (Elrefaie et al., 2024b), Aircraft (Luo et al., 2025), and CFD-VOL proposed by us. The datasets used in this study encompass a diverse range of surface geometries, each presenting unique structural challenges for the ML model. The heat sink dataset serves as a baseline case, consisting of geometries with sharp edges formed by combinations of 3D rectangular volumes. This dataset verifies the model performance when clear geometry tokens are present, as the algorithm can easily decompose these surfaces into flat facets. In contrast, the JEB dataset consists of non-parametric geometries created by generative models. These organic shapes pose a significant challenge for the model in extracting representative geometry tokens. The aircraft dataset introduces the challenge of representing smooth surfaces that lack distinct sharp edges. Finally, the DrivAer++ dataset includes vehicle models with a high number of individual components, serving as a benchmark for the model to handle complex, multi-component assemblies.

Prediction tasks also vary across these datasets, presenting different field-specific challenges. The heat sink and JEB datasets require the model to predict volumetric data, whereas the DrivAer++ dataset focuses on surface-based predictions. The aircraft dataset similarly involves surface prediction but adds the complexity of handling parametric

controls within the physics simulation. Finally, the proposed CFD-VOL dataset presents the unique challenge of predicting physics features in the external domain surrounding the geometry. This requires the model to accurately capture fluid patterns and field interactions outside the physical boundaries of the object.

Our geometry decomposition algorithm is applied exclusively to the manifold surface. We keep all the extracted tokens of heat sink dataset and the largest $M=128$ tokens for all the other datasets. Empirical benchmarks across all datasets indicate that the pre-processing time remains under 15 seconds per sample with CPU parallel computation. This negligible overhead ensures that the deterministic encoding phase does not become a bottleneck, maintaining the overall efficiency of the PGD-NO pipeline for rapid industrial design iterations. The visualization of the geometry tokens and predicted physics features are shown in Figure 3, and more extracted geometry tokens can be visualized in Appendix D

Baseline: We used the following model architectures for comparison: PointNet (Kashefi & Mukerji, 2022), MeshGraphNet (Pfaff et al., 2021), Graph Neural Operator (Anandkumar et al., 2020), GALERKIN (Cao, 2021), Geometry-informed Neural Operator (Li et al., 2023a), General Neural Operator Transformer (Hao et al., 2023), Transolver (Wu et al., 2024), Transolver++ (Luo et al., 2025), and AB-UPT (Alkin et al., 2025). The details of the hyper-parameter choice of our baseline methods are documented in Appendix E. To accommodate memory-constrained baselines, the mesh is decomposed into sub-domains for independent inference, then concatenated to reconstruct the global prediction, following the implementations of (Luo et al., 2025).

Table 1. Comparative analysis of relative error (%) across 3D industrial benchmarks. Bold values denote the best performance. “-” represents cases where the mesh resolution exceeds the model’s memory capacity or the architecture does not support the specific prediction task. In addition to global field errors, we report the prediction accuracy for critical physical measurements (Quantities of Interest) specific to each engineering design problem.

MODEL	HEAT SINK		JEB		DATA SET DRIVERNET++		AIRCRAFT		CFD-VOL	
	FIELD	$\bar{T}_{\text{CHIP}}$	FIELD	σ_{MAX}	FIELD	C_D	FIELD	C_L	FIELD	v_{MID}
POINTNET	5.21	7.54	40.1	55.4	33.4	23.7	15.2	9.15	24.5	26.2
MESHGRAPHNET	3.88	4.65	55.3	62.1	38.2	28.5	10.2	3.42	-	-
GNO	4.02	4.74	61.1	68.3	43.8	30.2	11.4	3.08	-	-
GALERKIN	3.52	5.15	44.2	48.9	26.4	22.1	10.1	6.53	-	-
GINO	2.68	4.19	38.9	44.2	28.8	24.5	11.9	4.32	-	-
GNOT	3.42	5.06	41.8	47.5	21.5	22.8	8.11	3.08	-	-
TRANSOLVER	3.20	4.89	42.6	46.3	20.1	20.3	5.23	1.41	-	-
AB-UPT	3.13	4.68	42.8	46.0	19.8	19.9	5.02	1.38	14.7	15.6
TRANSOLVER++	3.09	4.57	40.3	45.2	19.2	18.5	4.98	1.35	15.8	16.7
PGD-NO-v1 (OURS)	2.47	3.35	39.4	43.1	18.2	16.4	4.77	1.32	12.2	13.2
PGD-NO-v2 (OURS)	2.53	3.41	37.2	41.7	18.9	16.9	5.37	1.39	13.9	15.3
PGD-NO-v3 (OURS)	-	-	-	-	18.5	16.3	4.66	1.31	-	-

Metric: To quantify the accuracy of our model predictions, we employ the L^2 relative error as the primary metric. This provides a holistic assessment of the model’s performance over the entire spatial domain. However, global error metrics can sometimes obscure localized physical phenomena that are critical for engineering applications. Therefore, we supplement our analysis by reporting the prediction accuracy for specific critical quantities of interest (QoIs) tailored to each design problem. More details of the metric are clarified in Appendix F.

4.1. Main Results

The experimental results, summarized in Table 1, demonstrate that the proposed PGD-NO model consistently outperforms or achieves a similar level of accuracy as established baselines across all five 3D industrial benchmarks. Our architecture achieves the lowest relative errors in both volumetric field predictions and critical global scalar metrics. The performance improvement of PGD-NO over the second-best baseline is most significant on the heat sink dataset. This indicates that when the algorithm can perfectly decompose the geometry into representative tokens, the model is able to learn the underlying partial differential equations (PDEs) much more effectively. Our results on the JEB dataset also indicate that the algorithm is capable of handling distributionally diverse data. However, the improvement over other models in this case is more limited, which suggests a performance plateau when working with such irregular geometries.

Comparing PGD-NO-v1 and v2 reveals that adding an attention layer to geometry tokens primarily benefits datasets like JEB, where complex geometries are harder to partition precisely; otherwise, the extra complexity may introduce

noise and degrade performance. Conversely, PGD-NO-v3 hierarchical mapping—which omits long-range dependencies—yields the best results on the Aircraft dataset, suggesting long-range feature relationship are less effective than local relationship for the simple and smooth surfaces. Finally, PGD-NO strong performance on the CFD-VOL dataset demonstrates that surface-derived geometry tokens sufficiently capture the latent representations required for high-fidelity fluid simulations. More results are documented in Appendix C, including the computational and memory efficiency of different models and comparison with grid-pooling tokens. The error map comparison between our model and Transolver++ are provided in Figure 4, which is further clarified in Appendix G.

4.2. Larger Meshes

To evaluate the scalability of our proposed model on large-scale meshes, we increased the resolution of the CFD-VOL dataset using random spatial sampling and 3D interpolation. As shown in Table 2, we then compared the performance of Transolver++ and PGD-NO-v1 on this up-sampled data.

Table 2. Comparative relative error (%) on CFD-VOL across varying mesh resolutions and spatial sampling ratios.

Vol. points size	12M			60M		
	0.01	0.1	1.0	0.01	0.1	1.0
Transolver++	20.8	17.3	15.8	22.3	18.1	-
PGD-NO-v1	13.3	12.9	12.2	13.0	12.7	12.5

While Transolver++ fails to process the full 60M mesh due to memory constraints, PGD-NO-v1 maintains high accuracy even at this scale. To address the challenges of

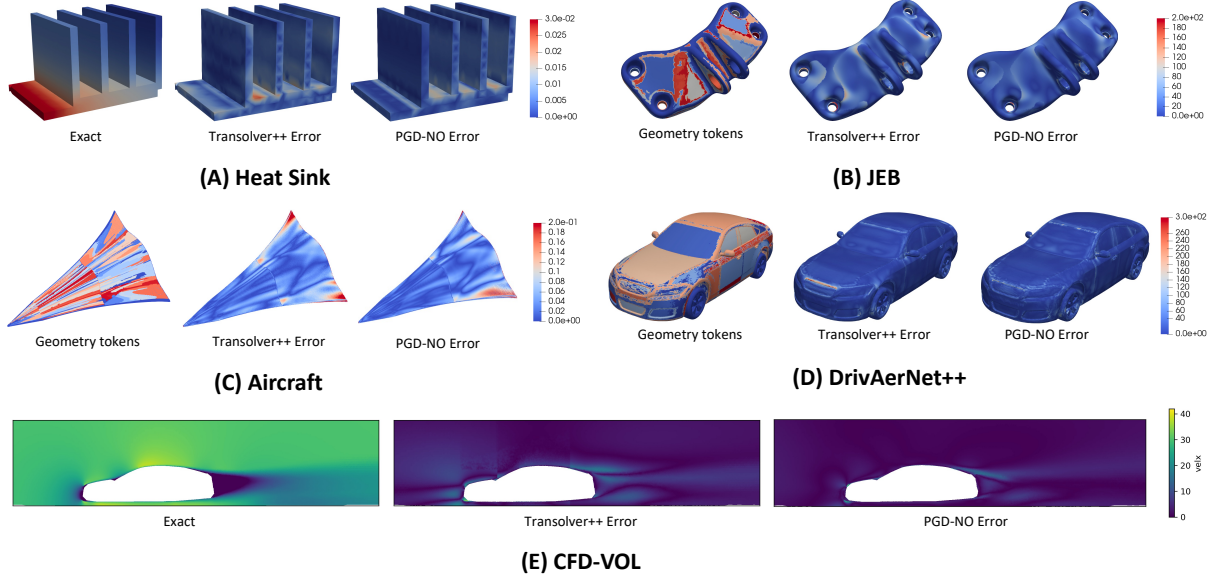


Figure 4. Error map comparison between Transolver++ and PGD-NO. Geometric references on the left provide spatial context for the residuals. For the Heat Sink and CFD-VOL datasets, we visualize the direct physical fields due to their specific structural or external domain requirements. For all other benchmarks, error maps are overlaid onto the extracted geometry tokens to demonstrate localized model accuracy across distinct structural regions.

huge dense point clouds, we also utilize a spatial random sampling strategy during training. By computing the loss gradient on a subset of points rather than the entire mesh, we significantly accelerate the training process and reduce peak memory requirements. Our results demonstrate that, Compared with Transolver++, this stochastic sampling approach does not significantly degrade the gradient quality of our model, as the model continues to deliver high prediction accuracy. Notably, as the total point cloud size increases, the performance gap caused by random sampling is further reduced.

4.3. Ablation Studies

Ablation studies on the CFD-VOL dataset (Table 3) compare the depth sensitivity of Transolver++ and PGD-NO-v1. While Transolver++ shows low sensitivity to increasing layers—likely because its adaptive physics tokens capture complex geometry tokens efficiently in shallower architectures. In contrast, the performance of our model decreases significantly if the number of Geometry Token Layers is insufficient. However, as the number of layers increases, our model demonstrates a superior ability to capture complex geometric features, leading to a consistent and significant enhancement in prediction accuracy. This suggests that the iterative refinement of geometry tokens is vital for improving our model performance.

We conduct an additional ablation study of number of tokens using the JEB dataset, as it contains the most complex

Table 3. Comparative L_2 relative error (%) on CFD-VOL across varying model depths.

Number of Layers	2	4	8	12
Transolver++	17.8	16.5	15.8	15.1
PGD-NO-v1	17.9	13.5	12.2	11.1

and organic geometries in our benchmark. As shown in Table 4, while performance degrades significantly with an insufficient number of tokens, marginal gains diminish as the token count scales up, which indicates that our proposed method can provide stable performance as sufficient geometry tokens are extracted.

Table 4. Comparative L_2 relative error (%) on JEB across varying number of extracted geometry tokens.

Number of Tokens	32	64	128	256
PGD-NO (Ours)	65.4	41.5	37.2	35.4

4.4. Model Explanation

Our model produces granular, node-wise importance maps by redistributing cross-attention weights from the Geometric Token Layers to their constituent nodes—a diagnostic capability absent in existing baseline models. In Figure 5, we visualize the nodal attention score distribution relative to the predicted location of maximum stress, with the peak attention region highlighted by a red arrow. The visualiza-

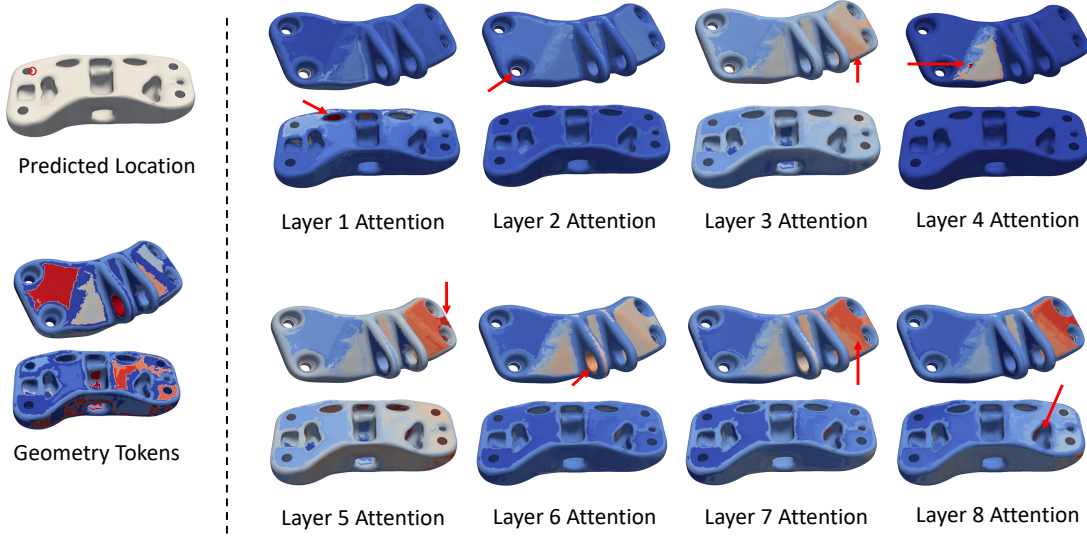


Figure 5. Interpretability analysis via multi-layer nodal importance mapping. (Left) Geometry reference showing the pre-computed tokens and the specific query location (red circle). (Right) Corresponding nodal importance maps of different geometry token layers, with red arrows highlighting the regions of peak attention.

tion reveals several key behaviors of the model. First, the model focuses on localized tokens near the predicted region before attending to larger, distant tokens. Generally, nodes on the left side of the bracket, which are closer to the region of interest, receive higher attention scores than those on the right. Notably, the model prioritizes the constraint region in Layer 2 and the loading region in Layer 6. This aligns with physical intuition, as these areas define the boundary conditions that govern the stress distribution. Each layer targets distinct spatial locations, indicating that the model successfully integrates multi-scale geometric details to inform its final prediction. These high-attention regions can ultimately serve as a design heuristic, guiding engineers toward critical structural areas that require optimization.

5. Conclusion

We introduce the Pre-computed Geometry Decomposition Neural Operator (PGD-NO), a paradigm that mitigates GPU memory bottlenecks in high-fidelity PDE simulations by shifting geometric encoding to a deterministic, pre-computed step. By extracting “geometry tokens” that effectively represent latent physical features, our model achieves linear scalability and handles massive meshes—up to 100 million nodes without inter-GPU communication overhead. Our results across five diverse industrial benchmarks demonstrate that PGD-NO can achieve similar accuracy as state-of-the-art baselines and provides enhanced interpretability.

Limitations: While PGD-NO matches state-of-the-art accu-

racy and offers enhanced interpretability via attention-based importance scores, combining our structural decomposition with local learnable refinements can be further investigated to better represent irregular, smooth topologies while maintaining the efficiency of decoupled volume-surface querying. Additionally, integrating recent advanced point cloud sampling techniques into our framework could further boost computational efficiency. Finally, because PGD-NO relies on precomputed geometry tokens, it exhibits robust invariance to underlying geometry representation. This decoupled nature makes our approach a promising candidate for building scientific foundation models capable of handling diverse data sources with varying geometric design principles and meshing formats—an avenue we intend to investigate thoroughly in the future.

Impact Statement

This paper presents work aimed at advancing the efficiency of neural operators for 3D physics deep learning. Our method enables large-scale physical simulations on complex industrial geometries that were previously computationally prohibitive, offering a generalizable surrogate modeling framework applicable to most current industry-scale modeling problems. Beyond accelerating industrial design and manufacturing, our focus on 3D geometry learning provides valuable insights for research into the development of geometry generation models and other frameworks centered on 3D geometries.

References

- Alkin, B., Fürst, A., Schmid, S., Gruber, L., Holzleitner, M., and Brandstetter, J. Universal physics transformers: A framework for efficiently scaling neural operators. *Advances in Neural Information Processing Systems*, 37: 25152–25194, 2024.
- Alkin, B., Bleeker, M., Kurl, R., Kronlachner, T., Sonnleitner, R., Dorfer, M., and Brandstetter, J. AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=nwQ8nitl1TZ>.
- Anandkumar, A., Azizzadenesheli, K., Bhattacharya, K., Kovachki, N., Li, Z., Liu, B., and Stuart, A. Neural operator: Graph kernel network for partial differential equations. In *ICLR 2020 workshop on integration of deep neural models and differential equations*, 2020.
- Bock, S. and Weiß, M. A proof of local convergence for the adam optimizer. In *2019 international joint conference on neural networks (IJCNN)*, pp. 1–8. IEEE, 2019.
- Cao, S. Choose a transformer: Fourier or galerkin. *Advances in neural information processing systems*, 34: 24924–24940, 2021.
- Eliasof, M., Haber, E., and Treister, E. Pde-gcn: Novel architectures for graph neural networks motivated by partial differential equations. *Advances in neural information processing systems*, 34:3836–3849, 2021.
- Elrefaie, M., Dai, A., and Ahmed, F. Drivaernet: A parametric car dataset for data-driven aerodynamic design and graph-based drag prediction. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 88360, pp. V03AT03A019. American Society of Mechanical Engineers, 2024a.
- Elrefaie, M., Morar, F., Dai, A., and Ahmed, F. Drivaernet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks. *Advances in Neural Information Processing Systems*, 37:499–536, 2024b.
- Gladstone, R. J., Rahmani, H., Suryakumar, V., Meidani, H., D’Elia, M., and Zareei, A. Mesh-based gnn surrogates for time-independent pdes. *Scientific reports*, 14(1):3394, 2024.
- Hao, Z., Wang, Z., Su, H., Ying, C., Dong, Y., Liu, S., Cheng, Z., Song, J., and Zhu, J. Gnot: A general neural operator transformer for operator learning. In *International Conference on Machine Learning*, pp. 12556–12569. PMLR, 2023.
- He, J., Kushwaha, S., Park, J., Koric, S., Abueidda, D., and Jasiuk, I. Sequential deep operator networks (s-deeponet) for predicting full-field solutions under time-dependent loads. *Engineering Applications of Artificial Intelligence*, 127:107258, 2024.
- Hisada, T. Recent progress in nonlinear fem-based sensitivity analysis. *JSME international journal. Ser. A, Mechanics and material engineering*, 38(3):301–310, 1995.
- Hong, S., Kwon, Y., Shin, D., Park, J., and Kang, N. Deep-jeb: 3d deep learning-based synthetic jet engine bracket dataset. *Journal of Mechanical Design*, 147(4), 2025.
- Jagota, V., Sethi, A. P. S., and Kumar, K. Finite element method: an overview. *Walailak Journal of Science and Technology (WJST)*, 10(1):1–8, 2013.
- Jermann, C., Trombettoni, G., Neveu, B., and Mathis, P. Decomposition of geometric constraint systems: a survey. *International Journal of Computational Geometry & Applications*, 16(05n06):379–414, 2006.
- Kashefi, A. and Mukerji, T. Physics-informed pointnet: A deep learning solver for steady-state incompressible flows and thermal fields on multiple sets of irregular geometries. *Journal of Computational Physics*, 468:111510, 2022.
- Lee, S. and Oh, T. Inducing point operator transformer: A flexible and scalable architecture for solving pdes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 153–161, 2024.
- Lévy, B. Laplace-beltrami eigenfunctions towards an algorithm that “understands” geometry. In *IEEE International Conference on Shape Modeling and Applications 2006 (SMI’06)*, pp. 13–13. IEEE, 2006.
- Li, K. and Ye, W. D-fno: A decomposed fourier neural operator for large-scale parametric partial differential equations. *Computer Methods in Applied Mechanics and Engineering*, 436:117732, 2025.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Stuart, A., Bhattacharya, K., and Anandkumar, A. Multipole graph neural operator for parametric partial differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020.
- Li, Z., Kovachki, N. B., and Azizzadenesheli, K. Burigede liu, kaushik bhattacharya, andrew stuart, and anima anandkumar. fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, volume 2, pp. 8, 2021.
- Li, Z., Kovachki, N., Choy, C., Li, B., Kossai, J., Otta, S., Nabian, M. A., Stadler, M., Hundt, C., Azizzadenesheli, K., et al. Geometry-informed neural operator

- for large-scale 3d pdes. *Advances in Neural Information Processing Systems*, 36:35836–35854, 2023a.
- Li, Z., Shu, D., and Barati Farimani, A. Scalable transformer for pde surrogate modeling. *Advances in Neural Information Processing Systems*, 36:28010–28039, 2023b.
- Li, Z., Zheng, H., Kovachki, N., Jin, D., Chen, H., Liu, B., Azizzadenesheli, K., and Anandkumar, A. Physics-informed neural operator for learning partial differential equations. *ACM/JMS Journal of Data Science*, 1(3):1–27, 2024.
- Lien, J.-M. and Amato, N. M. Approximate convex decomposition of polyhedra and its applications. *Computer Aided Geometric Design*, 25(7):503–522, 2008.
- Liu, N., Jafarzadeh, S., and Yu, Y. Domain agnostic fourier neural operators. *Advances in Neural Information Processing Systems*, 36:47438–47450, 2023.
- Lone, S. N., De, S., and Nayek, R. α -vi deeponet: A prior-robust variational bayesian approach for enhancing deeponets with uncertainty quantification. *Computer Methods in Applied Mechanics and Engineering*, 449: 118552, 2026.
- Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021a.
- Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021b.
- Lu, L., Meng, X., Cai, S., Mao, Z., Goswami, S., Zhang, Z., and Karniadakis, G. E. A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.
- Luo, H., Wu, H., Zhou, H., Xing, L., Di, Y., Wang, J., and Long, M. Transolver++: An accurate neural solver for PDEs on million-scale geometries. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 41432–41449. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/luo25o.html>.
- Mamou, K. and Ghorbel, F. A simple and efficient approach for 3d mesh approximate convex decomposition. In *2009 16th IEEE international conference on image processing (ICIP)*, pp. 3501–3504. IEEE, 2009.
- Mandl, L., Goswami, S., Lambers, L., and Ricken, T. Separable physics-informed deeponet: Breaking the curse of dimensionality in physics-informed machine learning. *Computer Methods in Applied Mechanics and Engineering*, 434:117586, 2025.
- Morrison, O. M., Pichi, F., and Hesthaven, J. S. Gfn: A graph feedforward network for resolution-invariant reduced operator learning in multifidelity applications. *Computer Methods in Applied Mechanics and Engineering*, 432:117458, 2024.
- Moukalled, F., Mangani, L., and Darwish, M. The finite volume method. In *The finite volume method in computational fluid dynamics: An advanced introduction with OpenFOAM® and Matlab*, pp. 103–135. Springer, 2015.
- Park, J. and Kang, N. Point-deeponet: Predicting nonlinear fields on non-parametric geometries under variable load conditions. *Neural Networks*, pp. 108560, 2026.
- Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., and Battaglia, P. W. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations (ICLR)*, 2021.
- Qiu, Y., Bridges, N., and Chen, P. Derivative-enhanced deep operator network. *Advances in Neural Information Processing Systems*, 37:20945–20981, 2024.
- Sahebi, A., Procaccini, M., and Giorgi, R. Hashgrid: an optimized architecture for accelerating graph computing on fpgas. *Future Generation Computer Systems*, 162: 107497, 2025.
- Serrano, L., Le Boudec, L., Kassai Koupaï, A., Wang, T. X., Yin, Y., Vittaut, J.-N., and Gallinari, P. Operator learning with neural fields: Tackling pdes on general geometries. *Advances in Neural Information Processing Systems*, 36: 70581–70611, 2023.
- Szuszik, C., Szatmári, R., Pál, G., and Kun, F. From segmentation to shattering: Structural transitions in the breakup of brittle rings. *Chaos, Solitons & Fractals*, 202:117573, 2026.
- Velasco, M., O’Hare, K., Rychtenberg, B., and Villar, S. Graph neural networks and non-commuting operators. *Advances in neural information processing systems*, 37: 95662–95691, 2024.
- Wang, T. and Wang, C. Latent neural operator for solving forward and inverse pde problems. *Advances in Neural Information Processing Systems*, 37:33085–33107, 2024.
- Wei, Y., Huang, Y. C., Tang, H., Sankaran, N., Chadha, I., Dai, D., Oluwole, O., Balan, V., and Lee, E. 9.3 nvlinc2c: A coherent off package chip-to-chip interconnect

- with 40gbps/pin single-ended signaling. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 160–162. IEEE, 2023.
- Wu, H., Luo, H., Wang, H., Wang, J., and Long, M. Transolver: A fast transformer solver for PDEs on general geometries. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 53681–53705. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/wu24r.html>.
- Yu, Y., Liu, N., Lu, F., Gao, T., Jafarzadeh, S., and Silling, S. A. Nonlocal attention operator: Materializing hidden knowledge towards interpretable physics discovery. *Advances in Neural Information Processing Systems*, 37: 113797–113822, 2024.
- Zhang, Z., Moya, C., Lu, L., Lin, G., and Schaeffer, H. Deeponet as a multi-operator extrapolation model: Distributed pretraining with physics-informed fine-tuning. *Journal of Computational Physics*, pp. 114537, 2025.
- Zheng, J., Li, W., Xu, N., Zhu, J., and Zhang, X. Alias-free mamba neural operator. *Advances in Neural Information Processing Systems*, 37:52962–52995, 2024.
- Zhong, W. Heatsink, 2025. URL <https://doi.org/10.7910/DVN/OZQMD4>.
- Zhong, W. and Meidani, H. Pi-vae: Physics-informed variational auto-encoder for stochastic differential equations. *Computer Methods in Applied Mechanics and Engineering*, 403:115664, 2023.
- Zhong, W. and Meidani, H. Physics-informed discretization-independent deep compositional operator network. *Computer Methods in Applied Mechanics and Engineering*, 431:117274, 2024.
- Zhong, W. and Meidani, H. Physics-informed geometry-aware neural operator. *Computer Methods in Applied Mechanics and Engineering*, 434:117540, 2025.

A. Model complexity analysis

In the Transolver++ framework, each physics-based attention layer requires approximately eight $O(N)$ operations per volume node. Specifically, for every block, the model must perform an **input projection** of nodal features to the internal head dimension, followed by **two separate linear mappings** to determine the Gumbel-Softmax temperature and the slice membership weights. The membership assignment itself necessitates a **Softmax operation** across the entire node set. The core interaction involves **two Einstein summation steps**: one to aggregate N nodes into a reduced set of geometry tokens, and another to redistribute the processed token features back to the original N coordinates. Finally, the layer concludes with an **output linear projection** and a subsequent **MLP pass** that maps the hidden representation back to the nodal feature space. In contrast, our first two proposed attention layer architectures, as illustrated in Figure 2 (B), optimize this process by requiring only three primary operations: query projection, attention score computation, and the linear operation for final output in a Multi-Head Attention layer. This architectural effectively reduces the computational overhead by more than 50% while maintaining representative capacity.

Regarding memory complexity, our model shifts the primary bottleneck from the volume nodes to the surface nodes. While Transolver++ operates on the entire point cloud holistically, resulting in a memory footprint dominated by the number of volume nodes ($O(8N_v)$), our framework decouples geometry feature extraction from solution querying. Because volume nodes in our model serve strictly as query points for PDE evaluation and do not participate in surface feature encoding, the volume point cloud can be partitioned and processed independently. In an extreme parallelization scenario, each volume node could theoretically be processed by a discrete unit. Consequently, the memory bottleneck resides within the surface node operations of the Geometry Token Layer. With a memory complexity of $O(3N_s)$, and utilizing hardware such as the NVIDIA GH200, our framework supports efficient parallel training for high-fidelity meshes containing up to 80 million surface nodes.

B. Dataset description

The details of the physics simulation are clarified:

- **Heat Sink (Thermal Management):** In the semiconductor industry, precise thermal management is essential to prevent hardware failure due to overheating. This simulation models steady-state heat dissipation from a CPU through a parameterized geometry. The design is parameterized by $\mathbf{p} \in \mathbb{R}^4$ (fin height, thickness, spacing, and count) mapped to a 10k-node 3D coordinate mesh $\mathcal{X}$. The output is the steady-state temperature field $T(\mathbf{x})$, governed by the Poisson equation for heat conduction:

$$-\nabla \cdot (\kappa \nabla T) = Q$$

where κ is the thermal conductivity and Q is the heat source. Boundary conditions specify forced convection $q = h(T - T_\infty)$ on the lateral surfaces and free convection on the top surface.

- **JEB (Aerospace Structural Engineering):** Reducing component weight while maintaining structural integrity is a core objective in aerospace manufacturing. This dataset utilizes 2,138 freeform 3D bracket designs where the input is a diverse set of 100k-scale meshes $\mathcal{M}$ with fixed bolt-hole constraints. The output is the displacement vector field $\mathbf{u}(\mathbf{x})$ and the stress distribution, derived from the linear elasticity equilibrium equation:

$$\nabla \cdot \sigma + \mathbf{f} = 0$$

where σ is the stress tensor and $\mathbf{f}$ represents the applied vertical loading of 3.5 kN. This setup tests the model’s ability to solve for structural stability under prescribed mechanical loads.

- **DrivAer++ (Automotive Aerodynamics):** Minimizing aerodynamic drag is critical for automotive OEMs to meet fuel efficiency and emission standards. The input consists of high-fidelity surface meshes of realistic car designs, including complex features like wheels and underbodies. The output is the surface pressure p and wall shear stress τ_w fields, which are solutions to the Reynolds-Averaged Navier-Stokes (RANS) equations for turbulent flow:

$$\rho(\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \mu \nabla^2 \mathbf{u} + \nabla \cdot \tau^R$$

where τ^R is the Reynolds stress tensor. Boundary conditions include a defined inlet velocity and no-slip conditions on the vehicle surface.

- **Aircraft (Transonic Aerodynamics):** Aircraft design requires evaluating aerodynamic performance across varying flight conditions to ensure stability and efficiency. The input features include a 300,000-point surface mesh alongside flow conditions: Mach number M , angle of attack α , and sideslip angle β . The output is a multi-variable state vector including fluid density ρ , pressure p , and velocity components (u, v, w) , governed by the compressible Navier-Stokes equations:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

This benchmark captures the complex physics of transonic flow, specifically the formation and movement of shock waves on the airframe surfaces.

- **CFD-VOL (Industrial Fluid Mechanics):** Full volumetric fluid analysis is required for comprehensive engineering tasks such as HVAC design and internal flow modeling. The output of the CFD simulation of the fluid part is the 3D velocity field $\mathbf{u}(\mathbf{x})$ on a 12M-node-scale mesh and pressure field $p(\mathbf{x})$ throughout the entire volume, governed by the incompressible Navier-Stokes and continuity equations:

$$\rho(\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}, \quad \nabla \cdot \mathbf{u} = 0$$

This setup focuses on the extreme scalability and local conservation required in large-scale industrial fluid simulations.

C. Additional results

In addition to accuracy comparison, we provide a detailed comparison of training times and memory usage in Table 5. Entries with ”-” indicate memory requirements exceeding the a 4-way GH200 capacity (approx. 360GB).

Table 5. Comparison of Training Times and Memory Usage Across Datasets

Dataset	PointNet	GNO	MeshGraphNet	Galerkin	GINO	GNOT	Transolver++	PGD-NO (Ours)
Heatsink	800s	9,000s	8,450s	8,660s	9,200s	10,800s	7,800s	6,200s
JEB	670s	7,400s	7,800s	22,600s	19,400s	15,100s	14,500s	14,000s
Aircraft	1.6h (15G)	28.2h (-)	27.8h (-)	16.4h (-)	7.8h (-)	8.1h (89G)	6.2h (77G)	5.4h (42G)
Drivernet	2.1h (10G)	37.5h (-)	37.3h (-)	11.6h (89G)	12.9h (-)	10.2h (75G)	9.5h (62G)	8.8h (35GB)
CFD-VOL	13.2h (30G)	-	-	-	-	-	24.3h (345G)	22h (120G)

To demonstrate the importance of our geometry decomposition for model performance, we compare it against a baseline where geometry latent tokens are instead generated via mean-pooling of the point cloud features. Specifically, this baseline partitions the point cloud into uniform grid cells, and compute the average of the point features inside each cell and treat it as one geometry token. Then we use the same decoder structure as our PGD-NO-v1 to generate predictions. As shown in the Table 6, this grid-pooling approach consistently underperforms across all datasets in terms of L_2 relative error when compared to our proposed geometry decomposition (Surface token).

Table 6. Comparison between surface tokens and grid-pooling tokens

Method	Heatsink	JEB	Aircraft	Drivernet	CFD-VOL
Grid- token	9.43	62.5	35.4	10.5	36.7
Surface token (Ours)	2.47	37.2	18.2	4.66	12.2

D. Extracted geometry token visualizations

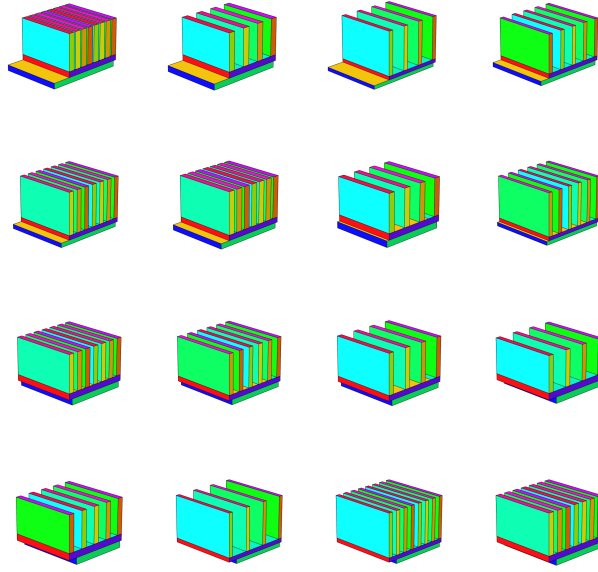
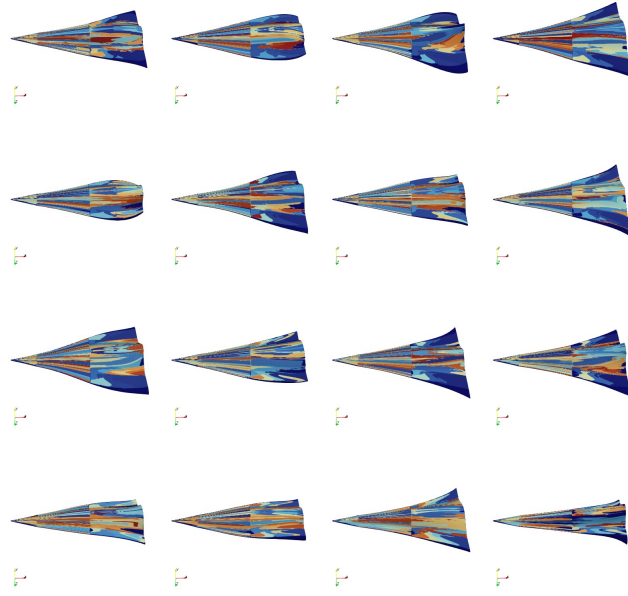


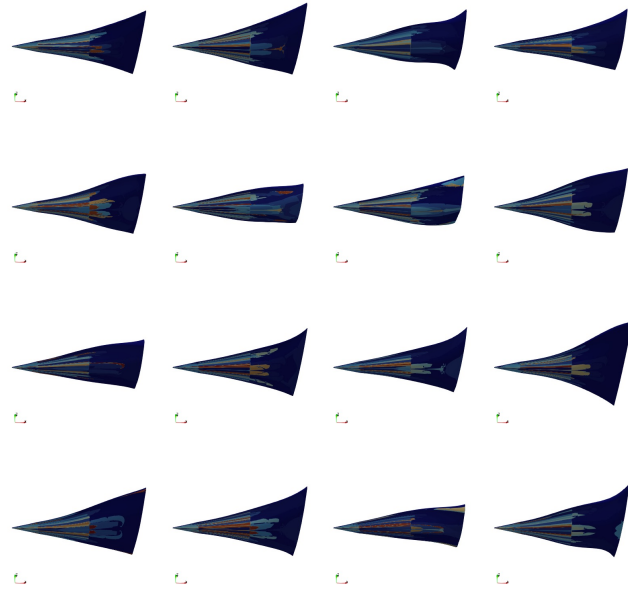
Figure 6. The extracted tokens of different samples in Heat Sink dataset are presented.



Figure 7. The extracted tokens of different samples in JEB dataset are presented.

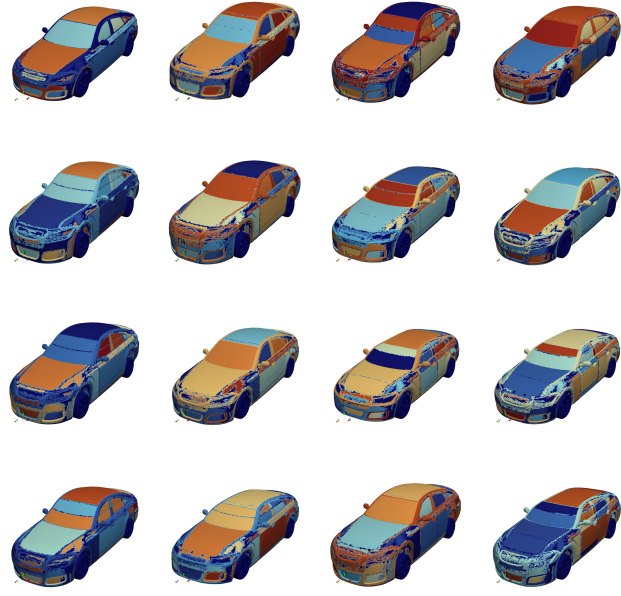


(a) Top view

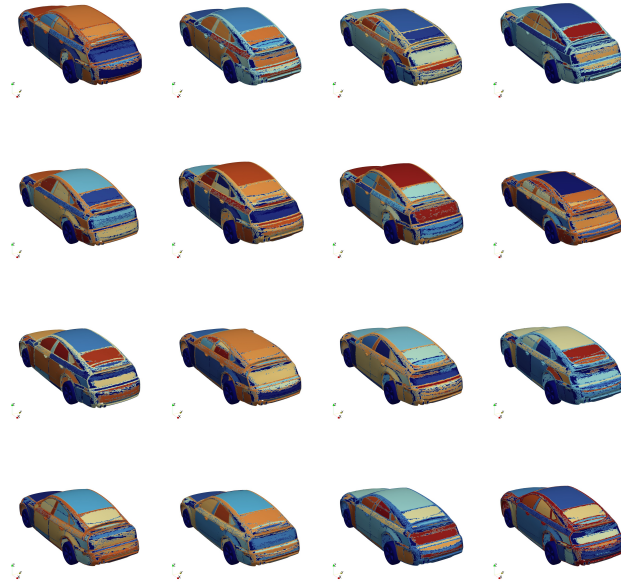


(b) Bottom view

Figure 8. The extracted tokens of different samples in JEB dataset are presented.



(a) Front view



(b) Back view

Figure 9. The extracted tokens of different samples in DrivAerNet++ dataset are presented.

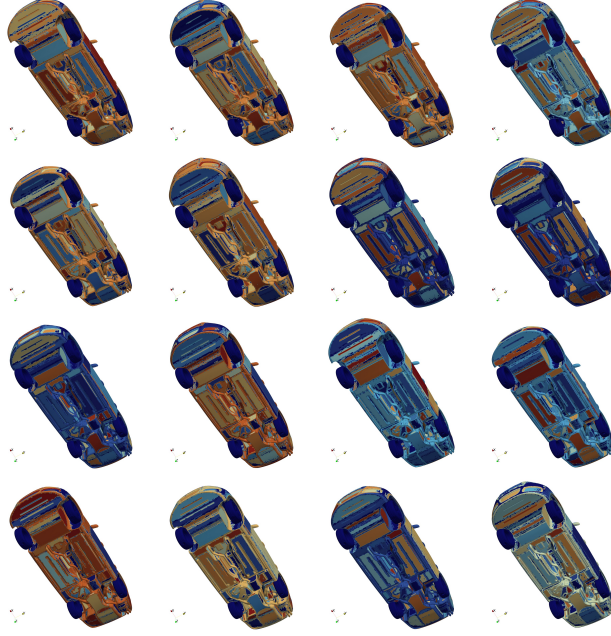


Figure 10. The bottom view of the extracted tokens of different samples in drivAerNet++ dataset are presented.

E. Implementation details and baseline comparison

To ensure a rigorous and fair comparison, all models are implemented within a unified experimental framework that standardizes architectural depth, hidden dimensions, and spatial resolution. Based on a combination of original implementations and GPU memory constraints, we set the default depth to 8 layers for point-based models and 4 layers for graph-based models. Hidden dimensions are assigned by model family: 128 for point-based networks (e.g., PointNet, Galerkin, GNOT, Transolver, Transolver++, and PGD-NO), 32 for graph-based operators (e.g., GNO, MeshGraphNet), and 16 for grid-based models (e.g., GINO). GELU is employed as the default activation function across all architectures. For point cloud inputs, graph connectivity is constructed using the KDTree algorithm, while grid-based models utilize a maximum hidden graph resolution of 100 voxels to maintain sufficient density.

To handle the extreme disparity in mesh scales—ranging from thousands to 2.5 million points—we implement a scale-aware adaptation strategy. For million-scale datasets such as DrivAerNet++ and Aircraft, where most baselines encounter memory limitations, we adopt a consistent subsampling approach. Inputs are reduced to 50k mesh points, with manifold structure preserved via K-Nearest Neighbors (KNN) reconstruction. During training, a random subsampling strategy is applied at the start of each epoch to increase data diversity while maintaining a constant step count. For evaluation, we aggregate multiple overlapping subsampled predictions into a single, high-fidelity reconstructed output. This ensures that even memory-constrained models can be evaluated on industry-scale geometries without compromising structural integrity.

To isolate architectural efficacy from optimization artifacts, all models are trained in a standardized environment using NVIDIA GH200 GPUs (100GB). We split each dataset into 60% training, 10% validation, and 30% testing sets. Training is conducted for 200 epochs with a batch size of 1, utilizing the Adam optimizer (Bock & Weiß, 2019) ($\beta_1 = 0.9, \beta_2 = 0.999$) and a weight decay of 10^{-4} . The base learning rate of 0.001 is managed by a OneCycleLR scheduler to facilitate stable convergence. In instances where a configuration exceeds available GPU memory, we reduce the hidden dimension to the largest size that fits the constraint.

By unifying these training-side variables, we ensure that the reported L_2 relative errors and Quantities of Interest (QoI) reflect the intrinsic predictive power of the model architectures rather than variations in hyperparameter tuning. Each training step computes gradients based on a single graph to maintain consistency across different geometric complexities. This rigorous protocol allows us to highlight the performance gains specifically attributable to the structural design of PGD-NO.

F. Metric

To evaluate the predictive performance of our model across diverse physical domains, we employ both a global field reconstruction metric and task-specific engineering measurements. This dual-layered evaluation ensures that the model captures the continuous solution space of the governing Partial Differential Equations (PDEs) while maintaining precision in the critical scalar values used for industrial design.

The primary metric for quantifying prediction accuracy is the L^2 relative error over the entire field. Specifically, we predict temperature for the heat sink dataset, stress for the JEB dataset, pressure for the Aircraft and DrivAerNet++ datasets, and velocity in the X-direction for the CFD-VOL dataset. To validate the model for specific design tasks, we also evaluate several critical physical quantities. For the heat sink dataset, we extract the temperature distributions for both the chip and the heat sink. We specifically monitor the average chip temperature, as it serves as a direct indicator of heat retention and cooling efficiency. For the JEB dataset, we focus on the accuracy of the maximum predicted stress to ensure the design remains within safe operating limits. In aerodynamic applications, we evaluate the DrivAer dataset using the drag coefficient (C_d) to measure wind resistance. For the aircraft dataset, we calculate the lift coefficient (C_l) to determine flight viability. Finally, for the volumetric CFD data, a single scalar value is often insufficient to characterize the fluid pattern. Consequently, we evaluate the fluid field predictions on the middle cross-section (v_{mid}), as this plane contains the most significant information regarding flow structures and boundary layer interactions.

F.1. Field prediction error

To assess the global accuracy of the predicted physical fields across the entire spatial domain, we utilize the **Relative L_2 Error**. This metric provides a normalized measure of the discrepancy between the model prediction $\hat{y}$ and the ground-truth field y , allowing for consistent comparison across different physical units and scales. It is defined as:

$$\text{Relative } L_2 = \frac{\|\hat{y} - y\|_2}{\|y\|_2} = \frac{\sqrt{\sum_{i=1}^n (\hat{y}_i - y_i)^2}}{\sqrt{\sum_{i=1}^n y_i^2}}. \quad (2)$$

This error is computed individually for each state variable in the output vector.

F.2. Quantities of Interest (QoI)

Beyond field-wide errors, we evaluate specific **Quantities of Interest (QoI)** that serve as the primary performance indicators in engineering design loops. These scalars are often derived through the integration or extremum searching of the predicted fields:

- **Heat Sink (Average Chip Temperature $\bar{T}_{\text{chip}}$):** The critical thermal performance metric is the average temperature at the contact interface between the heat source and the cooling fins:

$$\bar{T}_{\text{chip}} = \frac{1}{A_{\text{base}}} \int_{A_{\text{base}}} T(\mathbf{x}) dA \quad (3)$$

- **JEB (Maximum von Mises Stress σ_{max}):** For structural safety, we identify the peak stress concentration within the geometry to predict potential material yielding or structural failure:

$$\sigma_{\text{max}} = \max_{\mathbf{x} \in \mathcal{M}} \sigma_{vm}(\mathbf{x}) \quad (4)$$

- **DrivAer++ (Drag Coefficient C_D):** Automotive efficiency is measured by the drag coefficient, which requires integrating the predicted surface pressure p and wall shear stress τ_w in the direction of the flow:

$$C_D = \frac{2}{\rho v^2 A} \int_A (p \mathbf{n} \cdot \mathbf{i} + \tau_w \mathbf{t} \cdot \mathbf{i}) dA \quad (5)$$

- **Aircraft (Lift Coefficient C_L):** Aerodynamic performance for flight stability is determined by the lift coefficient, derived by integrating the pressure distribution over the wing surface along the vertical axis $\mathbf{k}$:

$$C_L = \frac{2}{\rho v^2 A} \int_A (p \mathbf{n} \cdot \mathbf{k}) dA \quad (6)$$

- **CFD-VOL (Cross-Cut Velocity Error Err_{cut}):** For large-scale volumetric flow, we evaluate the relative L_2 error of the velocity vector field $\mathbf{u}$ along specific 2D planes to ensure accurate capture of internal wake structures:

$$\text{Err}_{\text{cut}} = \frac{\|\hat{\mathbf{u}}_{\text{plane}} - \mathbf{u}_{\text{plane}}\|_2}{\|\mathbf{u}_{\text{plane}}\|_2} \quad (7)$$

G. Error map clarification

The visual analysis of the error distribution across diverse datasets highlights that the PGD-NO framework consistently achieves lower and more localized error patterns compared to the Transolver++ baseline. In thermal simulations like the Heat Sink, Transolver++ exhibits notable error clusters along the fin interfaces, whereas PGD-NO maintains a nearly uniform low-error profile across the entire geometry. This trend extends to complex structural and aerodynamic cases; for instance, in the JEB and Aircraft benchmarks, Transolver++ shows significant error concentrations in high-curvature regions and leading edges, while PGD-NO successfully mitigates these artifacts through its more effective geometric tokenization. In large-scale fluid dynamics simulations, such as the DrivAerNet++ surface and the corresponding velocity cross-sections, PGD-NO demonstrates superior precision in capturing boundary layer interactions and wake turbulences. By decoupling geometric feature extraction from point-wise querying, the model ensures that the prediction error remains minimal even in high-gradient areas, proving its robustness for high-fidelity industrial PDE modeling.